05_cts 스크립트는 Synopsys **IC Compiler II (ICC2)** 환경에서 가상의 클럭 네트워크를 실제 버퍼/인버터 트리 구조와 메탈 배선으로 구현하는 **Clock Tree Synthesis (CTS,** 클럭 트리 합성**)** 단계의 전체 Tcl 스크립트입니다.

배치(`refine_opt`) 단계의 데이터를 이어받아 실제 클럭 신호가 수만 개의 플립플롭에 최대한 동시에, 왜곡 없이 도달하도록 버퍼를 심고 타이밍을 보정하는 핵심 공정입니다.

---

1. 초기 환경 구성 및 라이브러리 핸들링

tcl

```
#made by kjinsoo
# module load
#module load icc2/2021.06-SP5
# open icc2_shell
#icc2_shell -gui | tee -i ./05_cts.log

#set current step & before step
set step cts
set before_step refine_opt

#source design_setup.tcl
source -e ./icc2_common_env/design_setup.tcl

#source timer.rpt script - report runtime for placement
source ./icc2_common_env/sec_runtime.tcl

sec_StartTimer copy_lib

#set_host_options -max_cores 16

#open lib
open_lib ../04_placement/nlib/ORCA_${before_step}.nlib
#copy lib
copy_lib -from ../04_placement/nlib/ORCA_${before_step}.nlib -to
./nlib/ORCA_${step}.nlib
#close & open current lib
close_lib -all
open_lib ./nlib/ORCA_${step}.nlib
#open block
open_block ./nlib/ORCA_${step}.nlib:ORCA.design
#link_block
link_block -rebind

sec_ReportResourceUtil copy_lib
```

코드를 사용할 때는 주의가 필요합니다.

- 해설: 앞선 `refine_opt` 단계의 최종 결과물 데이터베이스를 카피하여 `cts` 단계용 파일로 독립시킵니다. 런타임 측정 엔진(`sec_runtime.tcl`)을 기동하고, `link_block -rebind` 명령어로 디자인 내부 논리망과 물리 라이브러리를 새 데이터베이스 기준으로 동기화합니다.

---

2. MCMM 및 타이밍 제약 조건(SDC) 재정의

tcl

```
## Setup
####################################
#remove scenarios modes corners before setting
remove_scenarios -all
remove_modes -all
remove_corners -all

# Read TLU+ files
read_parasitic_tech -name {minTLU} -tlup $data(tech_tlupbc)
read_parasitic_tech -name {maxTLU} -tlup $data(tech_tlupwc)

# source mcmm(multi corners multi modes).tcl
source -e ./icc2_common_env/mcmm.tcl

#read_sdc(synopsys design constraints)
#scan_shift mode
current_mode scan_shift
read_sdc $data(shift_sdc)

#func mode
current_mode func
read_sdc $data(func_sdc)

#scan_capture mode
current_mode scan_capture
read_sdc $data(capture_sdc)
```

코드를 사용할 때는 주의가 필요합니다.

- 해설: 데이터의 일관성을 위해 기존 시나리오/코너 설정을 지우고 깨끗하게 리셋합니다. 파운드리의 RC 기생 성분 조회 파일인 TLU+(`minTLU`, `maxTLU`)와 전압/온도 환경 조건(`mcmm.tcl`)을 다시 정밀하게 로드합니다.

- **`read_sdc`**: 세 가지 동작 모드(`scan_shift`, `func`, `scan_capture`) 각각에 맞는 타이밍 제약 조건 파일들을 완전히 매핑하여 클럭 트리가 생성될 때 모든 모드의 제약 조건을 고루 만족하도록 준비합니다.

---

3. 클럭 특수 라우팅 규칙(NDR) 및 셀 사용 목적 강제 지정

tcl

```
#source cts_ex_ndr.tcl before cts
source -e ./icc2_common_env/cts_ex_ndr.tcl

get_scenario  -filter active&&hold
report_scenarios

#set hold & optimizaiton Cells
set_lib_cell_purpose -exclude hold [get_lib_cells]
set_lib_cell_purpose -include hold [get_lib_cells "*/NBUFFX2_HVT
saed32hvt/DELLN1X2_HVT"]

set_lib_cell_purpose -exclude optimization [get_lib_cells]
set_lib_cell_purpose -include optimization [get_lib_cells
"*/NBUFFX2_*VT */NBUFFX4_*VT */NBUFFX8_*VT */NBUFFX16_*VT"]
```

코드를 사용할 때는 주의가 필요합니다.

- **`cts_ex_ndr.tcl`**: 클럭 신호는 칩 전체에서 가장 자주, 빠르게 요동치므로 주변 신호선에 노이즈(Crosstalk)를 유발합니다. 이를 막기 위해 일반 배선보다 선 두께를 2배 넓히거나 간격을 넓게 벌려 깔도록 정의하는 **NDR(Non-Default Routing)** 규칙을 선언하는 중요한 스크립트입니다.
- **`set_lib_cell_purpose ... -exclude / -include hold`**: 툴이 홀드(Hold) 타이밍 위반을 고치기 위해 아무 셀이나 마구 삽입하는 것을 막기 위해 모든 셀을 제외시킨 뒤, 오직 고임계 전압 고유 지연 셀인 `NBUFFX2_HVT`와 `DELLN1X2_HVT` 등 특정 홀드 전용 버퍼만 사용하도록 강제 허용합니다.
- **`set_lib_cell_purpose ... optimization`**: 일반적인 타이밍 최적화(Setup 및 DRC 수정) 과정에서는 구동 능력이 검증된 크기별 버퍼 군(`NBUFFX2`, `X4`, `X8`, `X16`)만 골라서 사용하도록 통제하여 클럭 신호의 파형(Slew) 품질을 확보합니다.

---

4. 동작 시나리오 코너 활성화 및 가상 네트워크 해제

tcl

```
# Clock Tree Routing and IO latency update
```

```
####################################
# all about ss corner
set_scenario_status {*.ss*} -setup true -hold true \
                      -active true -max_transition true \
                      -max_capacitance true \
                      -min_capacitance false \
                        -leakage_power true \
                        -dynamic_power true
# all about ff corner
set_scenario_status {*.ff*} -setup true -hold true \
                             -active true -max_transition true \
                        -max_capacitance true \
                      -min_capacitance false \
                      -leakage_power true \
                           -dynamic_power true

# remove ideal network before cts
source -e ./user_script/remove_ideal_network_before_cts.tcl

# set_disable_timing
source -e ./user_script/set_disable_timing.tcl
```

코드를 사용할 때는 주의가 필요합니다.

- **`set_scenario_status`**: 최악(`*.ss*`) 및 최상(`*.ff*`) 동작 코너 전체에 걸쳐 타이밍, DRC, 전력 사양을 한꺼번에 최적화하도록 상태를 활성화합니다.
- **`remove_ideal_network_before_cts.tcl`**: 배치 단계에서 타이밍 연산 유연성을 위해 설정해 두었던 '지연 시간 **0**'의 가상 클럭 속성(**`ideal_network`**)을 완전히 걷어내는 핵심 필수 단계입니다. 이를 제거해야 비로소 툴이 실제 클럭 라우팅 배선과 버퍼를 집어넣기 시작합니다.

---

5. 모드별 목표 클럭 스큐(Skew) 및 지연 시간(Latency) 설정

tcl

```
#set target skew & latency constraints for clock trees
set mode "func scan_shift scan_capture"

foreach MODE $mode {
current_mode $MODE

echo "current mode is $MODE"
      switch $MODE {
            func {
```

```
                set_clock_tree_options -target_skew 0.3 -corners
[get_corners ss*] ;# Specifies the required value for maximum skew for
the specified clock trees (The default is 0)
                set_clock_tree_options -target_skew 0.3 -corners
[get_corners ff*]
                set_clock_tree_options -target_latency 0.5 -corners
[get_corners ss*] ;# Specifies the minimum early insertion delay
constraint for the specified clock trees (There is no default for this
option)
                set_clock_tree_options -target_latency 0.5 -corners
[get_corners ff*]
            }
        ... (scan_shift 및 scan_capture 모드 역시 동일하게 0.3 / 0.5로
설정) ...
      }
}

#report_clock_tree_options & check setting value
report_clock_tree_options
```

코드를 사용할 때는 주의가 필요합니다.

- **`set_clock_tree_options -target_skew 0.3`:** 칩 전체의 플립플롭들에 클럭 신호가 도달할 때, 가장 빨리 도착하는 곳과 가장 늦게 도착하는 곳의 최대 시간 차이**(Skew)**가 **0.3ns(300ps)**를 넘지 않도록 목표치를 제한합니다.
- **`-target_latency 0.5`:** 클럭 소스(시작점)로부터 최종 플립플롭 핀(끝점)까지 버퍼 체인을 타고 들어가는 기본 삽입 지연**(Insertion Delay)**을 **0.5ns** 수준으로 타겟팅합니다.
- **`report_clock_tree_options`:** 입력된 스큐 및 지연 타겟 값이 엔진에 올바르게 주입되었는지 리포트로 최종 확인합니다.

---

6. CTS 전용 어플리케이션 옵션(App Options) 튜닝

tcl

```
#setup for Non-ccd flow
#Non-ccd flow
set_app_options -name clock_opt.flow.enable_ccd -value false ;# This
app option controls whether the clock_opt command performs concurrent
clock and data (CCD) optimization (default : true)

#common setup for CTS
set_app_options -name opt.dft.clock_aware_scan_reorder -value true
;#This app option check hold violation status in scan chain first...
```

```
set_app_options -name time.remove_clock_reconvergence_pessimism -value
true ;# When this option is set to true, the tool removes clock
reconvergence pessimism from the slack calculation

#set max_fanout for CTS step
set_app_options -list {opt.tie_cell.max_fanout {16}}
set_app_options -list {cts.common.max_fanout {16}}
set_app_options -list {opt.common.max_fanout {16}}
```

코드를 사용할 때는 주의가 필요합니다.

- **`enable_ccd false`:** 클럭 트리 합성 시 클럭선과 데이터선을 동시에 꼬아가며 최적화하는 CCD(Concurrent Clock and Data) 기능을 비활성화(`false`)하고, 전통적이고 직관적인 순수 클럭 트리 빌드 플로우**(Non-CCD)**를 타도록 지시합니다.
- **`clock_aware_scan_reorder true`:** 테스트 스캔 체인 경로에서 발생할 수 있는 홀드 위반을 해결하기 위해, 클럭 드라이버의 위치를 고려하여 스캔 셀들의 연결 순서를 지리적으로 재정렬함으로써 홀드 보정용 버퍼 자원을 절약합니다.
- **`remove_clock_reconvergence_pessimism true`:** 동일한 클럭 소스에서 갈라져 나온 두 신호가 만날 때, 공통 클럭 경로에서 발생하는 툴 내부의 지나친 가상 타이밍 패널티를 제거하여 타이밍 여유 값**(Slack)** 계산의 정확도를 극대화**(CRPR** 활성화**)**합니다.
- **`max_fanout {16}`:** CTS 빌드 도중 버퍼나 일반 로직이 한 번에 구동할 수 있는 최대 문(Gate)의 개수를 16개로 제한하여 신호 왜곡을 방지합니다.

---

7. CTS 기동 전 사전 검사

tcl

```
#check before CTS
#check_clock_trees
check_clock_trees

#report_ideal_networks
report_ideal_network -modes {func scan_capture scan_shift} ;#if ideal
network exist, you must remove them
```

코드를 사용할 때는 주의가 필요합니다.

- **`check_clock_trees`:** 디자인 내에 클럭 소스가 명확히 정의되어 있는지, 끊어진 루프는 없는지 CTS 구동 전 구조적 결함을 검사합니다.
- **`report_ideal_network`:** 앞서 진행했던 `ideal_network` 제거 작업이 누락된 곳 없이 완벽히 처리되었는지 3개 모드 전체를 전수 조사합니다. (만약 가상

네트워크가 하나라도 남아있다면 CTS가 정상 동작하지 않으므로 에러를 잡아내는 단계입니다.)

---

8. 핵심 CTS 프로세스 실행 (`clock_opt`)

tcl

```
########################
#Running CTS
########################
# pre-CTS
sec_StartTimer cts

clock_opt -to route_clock

sec_ReportResourceUtil cts

save_lib -all
```

코드를 사용할 때는 주의가 필요합니다.

- **`clock_opt -to route_clock`**: 이 스크립트의 메인 사령탑 명령어입니다. 클럭 트리를 위한 버퍼/인버터 트리를 물리적으로 심고(`Synthesis`), 타이밍 스큐를 조율하기 위해 밸런싱 작업을 거친 뒤, 최종적으로 클럭 배선 자체를 실제 메탈선으로 완전히 라우팅(`route_clock`)해내는 마일스톤 단계입니다.
- 완료 후 진행 상황을 저장(`save_lib -all`)합니다.

---

9. CTS 결과 리포트 및 최종 저장 후 종료

tcl

```
#report after CTS
#update timing
update_timing -full ;#Updates timing information on the current design

#timing
report_timing -significant_digits 5 ;#Displays timing information about
a design

#Qor summary
report_qor -summary ;#Displays QoR information and statistics for the
current design

############################################################
```

```
#report Timing & constraints for CTS
############################################################
#update timing
update_timing -full ;#Updates timing information on the current design

sec_StartTimer report
#write "reports files" of placement at ${rundir}/reports/cts/*
directory
source ./user_script/cts_rpt.tcl
sec_ReportResourceUtil report

exit
```

코드를 사용할 때는 주의가 필요합니다.

- 해설**:** 클럭 트리가 완벽하게 고정(Physical Realization)되었으므로 칩 전체 타이밍 맵을 전면 새로고침(`update_timing -full`)합니다.
- **`report_timing / report_qor`:** 스큐 타겟(0.3ns)이 잘 지켜졌는지, 클럭이 정상적으로 심어진 후의 최종 QoR 성능 지표가 어떤지 확인합니다.
- **`cts_rpt.tcl`:** 지정된 리포트 폴더에 정밀 분석용 종합 CTS 성적표 파일을 백업 출력하고 쉘을 무사히 종료(`exit`)합니다. 이 단계가 끝나면 비로소 데이터는 최종 신호선 배선 단계인 **Routing (**라우팅**)** 단계로 넘어갈 준비를 마칩니다.